Lab Handout 1-D

Bidirectional DATA/ACK Ping-Pong Between Two Cooperative Targets

Capstone — Module 1: Radio Frequency (RF) Hardware Foundations

1 Objective

Implement bidirectional half-duplex LoRa communication between two STM32+RFM95W nodes. Target A transmits DATA packets on a 1 s schedule; Target B receives every DATA and immediately replies with an ACK whose sequence number echoes the received DATA. Both nodes log timestamps so that round-trip latency, packet error rate, and Received Signal Strength Indicator (RSSI)/Signal-to-Noise Ratio (SNR) can be measured. This handout establishes the bidirectional baseline that Lab Handout 1-F (cooperative target firmware with full Finite State Machine (FSM)) and Lab Handout 1-G (interceptor / Threat firmware) build on.

2 Prerequisites

- Lab Handouts 1-A, 1-B, 1-C complete for at least **two** nodes. The two projects `Nucleo_RFM95W_TargetA` and `Nucleo_RFM95W_TargetB` must already exist from LH1-C Step 1, and at least `TargetA` must have demonstrated `RegVersion = 0x12` on its Nucleo in LH1-C Step 5.

3 Materials

- 2 × assembled RFM95W breakouts.
- 2 × Nucleo-L476RG boards + USB cables.
- 2 × host USB ports (or a powered USB hub).
- Breadboards and jumpers for both nodes.

4 Procedure

4.1 Step 1 — Open the pre-existing TargetA and TargetB projects

The two projects `Nucleo_RFM95W_TargetA` and `Nucleo_RFM95W_TargetB` were created up front in Lab Handout 1-C, Step 1, with identical baseline peripheral configurations (SPI1, USART2, USART3 with DMA, TIM2 1 ms tick, GPIO labels for `RFM_CS`/`RFM_RST`/`RFM_DIO0`, EXTI on PA8). **Do not copy, rename, or move these folders** — moving STM32 project folders breaks the CMake cache, the launch.json paths, and the STM32 extension's project detection. Instead, open each in its own VS Code window and add code in place.

1. Open `Nucleo_RFM95W_TargetA` in VS Code: **File** → **Open Folder** → select the `Nucleo_RFM95W_TargetA` folder created in LH1-C. The folder must be the root of the VS Code window.
2. Press `F7` to confirm the project still builds. (At this point `TargetA` contains the LH1-C SPI-bring-up code, which will be removed in Step 2 below.)
3. Open `Nucleo_RFM95W_TargetB` in a **second VS Code window** so both projects are open simultaneously, each with its own Run-and-Debug panel and Serial Monitor.
4. Press `F7` in the second window to confirm `TargetB` builds. (`TargetB` is freshly generated from LH1-C with no application code beyond the CubeMX peripheral init, so it will build immediately with no warnings.)

Why two physically separate projects with hard-coded roles?

The cooperative targets (Target A and Target B) implement different firmware paths — Target A initiates DATA, Target B responds with ACK. Two physically separate projects let students keep both VS Code windows open, each pointing at a different Nucleo, and flash each one independently with a single **Start Debugging** click. In Lab Handout 1-F, both projects gain the full cooperative-target FSM (including the PAUSE listener path) and remain physically separate; the role distinction does not move to a runtime flag.

4.2 Step 2 — Add the shared LoRa initialization function

This step applies to **both** the `Nucleo_RFM95W_TargetA` and `Nucleo_RFM95W_TargetB` projects. Make the same edits in each project's `main.c`.

Both targets must use identical physical-layer parameters. Table 1 lists the Module 1 reference settings; these will also be used in Lab Handouts 1-F and 1-G.

Table 1: Baseline LoRa physical-layer configuration.

Parameter	**Value**	**Register / Value**
Carrier frequency	906.5 MHz	`RegFrfMsb=0xE2, Mid=0xA0, Lsb=0x00`
Spreading factor	SF7	`RegModemConfig2 = 0x74`
Bandwidth	125 kHz	`RegModemConfig1 = 0x72`
Coding rate	4/5	(combined into `RegModemConfig1 = 0x72`)
Header mode	Explicit	(combined into `RegModemConfig1 = 0x72`)
Cyclic Redundancy Check (CRC)	On	(combined into `RegModemConfig2 = 0x74`)
Preamble length	8 symbols	`RegPreambleMsb=0x00, Lsb=0x08`
Sync word	`0x34` (private)	`RegSyncWord = 0x34`
Output power	+14 dBm (PA_BOOST)	`RegPaConfig = 0x8C`
Payload length (TX)	32 bytes	`RegPayloadLength = 0x20`
TX FIFO base addr	—	`RegFifoTxBaseAddr = 0x80`
RX FIFO base addr	—	`RegFifoRxBaseAddr = 0x00`

Frequency Match is Critical

Both targets **must** write the same `RegFrf*` values. A frequency offset of even a few kHz will prevent reception at SF7. Copy the three bytes from Table 1 exactly — do not substitute 915.0 MHz or any other frequency.

1. In `main.c`, locate `/* USER CODE BEGIN PFP */` and add prototypes for every function you will define in this lab, alongside the existing LH1-C prototypes:

```
/* USER CODE BEGIN PFP */
int __io_putchar(int ch);
uint8_t rfm_read_reg(uint8_t addr);
void rfm_write_reg(uint8_t addr, uint8_t val);
uint32_t rfm_frf_to_hz(uint8_t msb, uint8_t mid, uint8_t lsb);
void lora_init(void);
void rfm_set_mode_rx(void);
void rfm_set_mode_standby(void);
void build_packet(uint8_t *buf, uint8_t sender_id, uint16_t seq,
                  uint8_t pkt_type, uint8_t mod_kind);
uint16_t crc16_ccitt(const uint8_t *data, int len);
/* USER CODE END PFP */
```

2. Scroll to `/* USER CODE BEGIN 4 */` near the bottom of `main.c` and add the `lora_init()` definition immediately after the existing helpers from LH1-C:

```
void lora_init(void) {
    /* 1. Enter LoRa SLEEP (required before writing FRF registers) */
    rfm_write_reg(0x01, 0x80);
    HAL_Delay(10);

    /* 2. Carrier frequency: 906.5 MHz */
    rfm_write_reg(0x06, 0xE2);  // RegFrfMsb
    rfm_write_reg(0x07, 0xA0);  // RegFrfMid
    rfm_write_reg(0x08, 0x00);  // RegFrfLsb

    /* 3. Modem config: BW=125 kHz, CR=4/5, explicit header */
    rfm_write_reg(0x1D, 0x72);

    /* 4. Modem config: SF7, single packet, CRC on */
    rfm_write_reg(0x1E, 0x74);

    /* 5. Preamble length: 8 symbols */
    rfm_write_reg(0x20, 0x00);
    rfm_write_reg(0x21, 0x08);

    /* 6. Sync word: 0x34 (private LoRa network) */
    rfm_write_reg(0x39, 0x34);

    /* 7. PA: PA_BOOST, +14 dBm */
    rfm_write_reg(0x09, 0x8C);

    /* 8. FIFO base addresses */
    rfm_write_reg(0x0E, 0x80);
    rfm_write_reg(0x0F, 0x00);
```

```
    /* 9. Return to STANDBY */
    rfm_write_reg(0x01, 0x81);
    HAL_Delay(10);

    printf("lora_init done\r\n");
}

void rfm_set_mode_standby(void) {
    rfm_write_reg(0x01, 0x81);  // LoRa STANDBY
}

void rfm_set_mode_rx(void) {
    /* Reset FIFO RX pointer and enter RXCONTINUOUS */
    rfm_write_reg(0x0D, rfm_read_reg(0x0F));  // RegFifoAddrPtr = RxBase
    rfm_write_reg(0x12, 0xFF);                 // Clear all IRQ flags
    rfm_write_reg(0x01, 0x85);                 // LoRa RXCONTINUOUS
}
/* USER CODE END 4 */
```

3. Add the `crc16_ccitt` helper inside the same `USER CODE BEGIN 4` block. The polynomial is `0x1021`, init `0xFFFF`, no reflect, no XOR-out (CCITT-FALSE):

```
uint16_t crc16_ccitt(const uint8_t *data, int len) {
    uint16_t crc = 0xFFFF;
    for (int i = 0; i < len; i++) {
        crc ^= ((uint16_t)data[i]) << 8;
        for (int b = 0; b < 8; b++) {
            crc = (crc & 0x8000) ? (uint16_t)((crc << 1) ^ 0x1021)
                                 : (uint16_t)(crc << 1);
        }
    }
    return crc;
}
```

4. Add the `build_packet` helper that constructs a 32-byte radio packet per the Module 1 packet format (Table 4 in the lesson plan). For LH1-D the payload region is filled with a fixed 17-byte test pattern; LH1-F will replace this with structured per-packet payloads.

```
void build_packet(uint8_t *buf, uint8_t sender_id, uint16_t seq,
                  uint8_t pkt_type, uint8_t mod_kind) {
    /* 0-1: Preamble sync 0xAE5A (big-endian) */
    buf[0] = 0xAE; buf[1] = 0x5A;
    /* 2: Sender Beacon ID */
    buf[2] = sender_id;
    /* 3-4: Sequence number (little-endian) */
    buf[3] = (uint8_t)(seq);
    buf[4] = (uint8_t)(seq >> 8);
    /* 5: TX power (dBm) */
    buf[5] = 14;
    /* 6: Modulation Code, packed: bits[7:2]=mod_kind<<2, bits[1:0]=pkt_type */
    buf[6] = (uint8_t)((mod_kind << 2) | (pkt_type & 0x03));
    /* 7: Spreading factor */
    buf[7] = 7;
    /* 8: Bandwidth code (0x00 = 125 kHz) */
    buf[8] = 0x00;
    /* 9-12: Timestamp */
    uint32_t ts = HAL_GetTick();
    buf[9]  = (uint8_t)(ts);
    buf[10] = (uint8_t)(ts >> 8);
    buf[11] = (uint8_t)(ts >> 16);
    buf[12] = (uint8_t)(ts >> 24);
    /* 13-29: Payload (fixed test pattern for LH1-D) */
    for (int i = 13; i < 30; i++) buf[i] = 0xA5;
    /* 30-31: CRC-16/CCITT-FALSE over bytes 0-29, MSB-first */
    uint16_t crc = crc16_ccitt(buf, 30);
    buf[30] = (uint8_t)(crc >> 8);
    buf[31] = (uint8_t)(crc);
}
```

5. Locate `/* USER CODE BEGIN 2 */`. The LH1-C reset sequence and `RegVersion` read are already there. Add a call to `lora_init()` immediately after the `RegVersion` print, and **remove or comment out** any LH1-C demo code that follows (the frequency write/read-back and the mode-transition demo). Final state of the block:

```
/* USER CODE BEGIN 2 */

/* Reset the RFM95W (from LH1-C) */
HAL_GPIO_WritePin(RFM_RST_GPIO_Port, RFM_RST_Pin, GPIO_PIN_RESET);
HAL_Delay(1);
HAL_GPIO_WritePin(RFM_RST_GPIO_Port, RFM_RST_Pin, GPIO_PIN_SET);
HAL_Delay(10);

/* Sanity check (from LH1-C) */
uint8_t version = rfm_read_reg(0x42);
printf("RegVersion = 0x%02X\r\n", version);

/* Configure the LoRa modem */
lora_init();

/* USER CODE END 2 */
```

6. Press `F7` in each VS Code window to build both projects. Confirm zero errors before continuing.

Packet-type and modulation-kind constants

For convenience and readability, define these constants near the top of `main.c` (in `/* USER CODE BEGIN PD */` or similar):

```
#define PKT_TYPE_DATA   0x00
#define PKT_TYPE_ACK    0x01
#define PKT_TYPE_PAUSE  0x02

#define MOD_KIND_LORA   0x01
#define MOD_KIND_FSK    0x02
#define MOD_KIND_OOK    0x03

#define BEACON_ID_A     0x01
#define BEACON_ID_B     0x02
#define BEACON_ID_THREAT 0xFF
```

4.3 Step 3 — Add Target A (DATA initiator + ACK listener) main-loop code

This step applies **only** to the `Nucleo_RFM95W_TargetA` project.

1. In `main.c` of the `Nucleo_RFM95W_TargetA` project, locate `/* USER CODE BEGIN 2 */` and add a sequence counter and timing variables immediately after the `lora_init()` call:

```
/* USER CODE BEGIN 2 */
/* ... reset, RegVersion read, lora_init() above ... */

uint16_t seq = 0;
uint32_t next_tx_ms = HAL_GetTick() + 1000;  // first DATA at t=1s
uint16_t acks_received = 0;
uint16_t data_sent = 0;

printf("Target A initiator ready\r\n");

/* USER CODE END 2 */
```

2. Locate `/* USER CODE BEGIN WHILE */` and add the per-iteration TX-then-RX-window code. Target A transmits a DATA packet, switches into RX, polls for an ACK with a 200 ms RX-window timeout, then logs the round-trip outcome:

```
/* USER CODE BEGIN WHILE */
while (1)
{
    if (HAL_GetTick() >= next_tx_ms) {
        next_tx_ms += 1000;  // schedule next DATA in 1 second

        /* --- Build and transmit DATA --- */
        uint8_t pkt[32];
        build_packet(pkt, BEACON_ID_A, seq,
                     PKT_TYPE_DATA, MOD_KIND_LORA);

        rfm_set_mode_standby();
        /* Point FIFO to TX base address */
        rfm_write_reg(0x0D, rfm_read_reg(0x0E));
        /* Write packet into FIFO */
        for (int i = 0; i < 32; i++) rfm_write_reg(0x00, pkt[i]);
        rfm_write_reg(0x22, 32);  // RegPayloadLength

        uint32_t t_tx_ms = HAL_GetTick();
        rfm_write_reg(0x12, 0xFF);  // Clear IRQ flags
        rfm_write_reg(0x01, 0x83);  // LoRa TX

        /* Wait for TxDone (IRQ flag bit 3) */
        uint8_t irq;
        do {
            irq = rfm_read_reg(0x12);
```

```
} while ((irq & 0x08) == 0);
rfm_write_reg(0x12, 0xFF);

data_sent++;

/* --- Switch to RX and wait up to 200 ms for an ACK --- */
rfm_set_mode_rx();
uint32_t rx_window_end = HAL_GetTick() + 200;
uint8_t got_ack = 0;
uint32_t t_ack_ms = 0;
uint16_t ack_seq = 0;
int16_t ack_rssi = 0;

while (HAL_GetTick() < rx_window_end) {
    irq = rfm_read_reg(0x12);
    if (irq & 0x40) {  /* RxDone */
        if ((irq & 0x20) == 0) {  /* CRC ok */
            uint8_t rx_addr = rfm_read_reg(0x10);
            rfm_write_reg(0x0D, rx_addr);
            uint8_t rx[32] = {0};
            for (int i = 0; i < 32; i++) {
                rx[i] = rfm_read_reg(0x00);
            }
            /* Validate sender = B and pkt_type = ACK */
            uint8_t pkt_type_rx = rx[6] & 0x03;
            if (rx[2] == BEACON_ID_B
                && pkt_type_rx == PKT_TYPE_ACK) {
                got_ack = 1;
                t_ack_ms = HAL_GetTick();
                ack_seq = (uint16_t)rx[3]
                        | ((uint16_t)rx[4] << 8);
                ack_rssi = (int16_t)rfm_read_reg(0x1A) - 157;
            }
        }
        rfm_write_reg(0x12, 0xFF);
        if (got_ack) break;
    }
}
rfm_set_mode_standby();

/* --- Log CSV: seq, t_tx_ms, t_ack_ms, rtt_ms, rssi_dbm --- */
if (got_ack) {
    acks_received++;
    uint32_t rtt = t_ack_ms - t_tx_ms;
    printf("CSV,%u,%lu,%lu,%lu,%d\r\n",
           (unsigned)seq, (unsigned long)t_tx_ms,
           (unsigned long)t_ack_ms, (unsigned long)rtt,
           (int)ack_rssi);
```

```
        } else {
            printf("CSV,%u,%lu,0,0,0  (no ACK; seq=%u)\r\n",
                   (unsigned)seq, (unsigned long)t_tx_ms,
                   (unsigned)seq);
        }

        if ((data_sent % 10) == 0) {
             uint32_t ack_x10 = (1000UL * acks_received) / data_sent;
             printf("# stats: sent=%u acked=%u (%lu.%lu%%)\r\n",
                 data_sent, acks_received,
                 ack_x10 / 10UL, ack_x10 % 10UL);
        }

        seq++;
    }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

The `USER CODE END WHILE` and `USER CODE BEGIN 3` markers are already present in the CubeMX-generated file — do not add or remove them. Your code goes *between* `USER CODE BEGIN WHILE` and `USER CODE END WHILE`.

3. Press `F7` to build. Confirm zero errors.

4.4 Step 4 — Add Target B (DATA listener + ACK responder) main-loop code

This step applies **only** to the `Nucleo_RFM95W_TargetB` project.

1. In `main.c` of the `Nucleo_RFM95W_TargetB` project, locate `/* USER CODE BEGIN 2 */` and immediately after `lora_init()` put the radio into continuous RX:

```
/* USER CODE BEGIN 2 */
/* ... reset, RegVersion read, lora_init() above ... */

rfm_set_mode_rx();
uint16_t data_rx = 0;
uint16_t acks_sent = 0;

printf("Target B responder ready\r\n");

/* USER CODE END 2 */
```

2. Locate `/* USER CODE BEGIN WHILE */` and add the per-iteration RX-then-ACK code:

```
/* USER CODE BEGIN WHILE */
while (1)
{
    uint8_t irq = rfm_read_reg(0x12);

    if (irq & 0x40) {  /* RxDone */
        if (irq & 0x20) {  /* CRC error */
            printf("CRC error\r\n");
            rfm_write_reg(0x12, 0xFF);
        } else {
            /* Read packet */
            uint8_t rx_addr = rfm_read_reg(0x10);
            rfm_write_reg(0x0D, rx_addr);
            uint8_t rx[32] = {0};
            for (int i = 0; i < 32; i++) {
                rx[i] = rfm_read_reg(0x00);
            }
            int16_t rssi = (int16_t)rfm_read_reg(0x1A) - 157;
            int8_t  snr_raw = (int8_t)rfm_read_reg(0x19);
            int16_t snr = snr_raw / 4;
            rfm_write_reg(0x12, 0xFF);  /* clear IRQs */

            uint8_t sender = rx[2];
            uint8_t pkt_type_rx = rx[6] & 0x03;
            uint16_t rx_seq = (uint16_t)rx[3]
                            | ((uint16_t)rx[4] << 8);

            if (sender == BEACON_ID_A
                && pkt_type_rx == PKT_TYPE_DATA) {
```

```
                data_rx++;
                printf("RX DATA seq=%u rssi=%d snr=%d\r\n",
                       (unsigned)rx_seq, (int)rssi, (int)snr);

                /* --- Build and TX ACK echoing the DATA's seq --- */
                uint8_t ack[32];
                build_packet(ack, BEACON_ID_B, rx_seq,
                             PKT_TYPE_ACK, MOD_KIND_LORA);

                rfm_set_mode_standby();
                rfm_write_reg(0x0D, rfm_read_reg(0x0E));
                for (int i = 0; i < 32; i++) {
                    rfm_write_reg(0x00, ack[i]);
                }
                rfm_write_reg(0x22, 32);
                rfm_write_reg(0x12, 0xFF);
                rfm_write_reg(0x01, 0x83);  /* LoRa TX */

                /* Wait for TxDone */
                do {
                    irq = rfm_read_reg(0x12);
                } while ((irq & 0x08) == 0);
                rfm_write_reg(0x12, 0xFF);

                acks_sent++;

                /* Return to RXCONTINUOUS for next DATA */
                rfm_set_mode_rx();
            }
        }
    }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
```

3. Press `F7` to build. Confirm zero errors.

4.5 Step 5 — Run the ping-pong end-to-end

1. Plug both Nucleos into the host PC via USB.
2. Open the Serial Monitor in each VS Code window using the procedure from LH1-C: Command Palette (`Ctrl+Shift+P`) → **Serial Monitor: Focus on Serial Monitor View** → **Open Serial Monitor** → select the STLink Virtual COM port → baud **115200** → **Start Monitoring**. Identify which COM port belongs to which Nucleo by unplugging one board and observing which port disappears from the dropdown.
3. In the Target B VS Code window, click **Start Debugging** first (so the responder is listening before the initiator starts). Wait for `Target B responder ready` on its Serial Monitor.
4. In the Target A VS Code window, click **Start Debugging**. The Target A monitor should begin printing one `CSV,...` line per second; the Target B monitor should print one `RX DATA seq=...` line per matching DATA reception.
5. Place the two Nucleos approximately 1 m apart on the bench with their spring antennas upright.

Expected steady-state output on Target A's monitor:

```
RegVersion = 0x12
lora_init done
Target A initiator ready
CSV,0,1041,1191,150,-40
CSV,1,2041,2191,150,-40
CSV,2,3041,3191,150,-39
...
# stats: sent=10 acked=10 (100.0%)
```

And on Target B's monitor:

```
RegVersion = 0x12
lora_init done
Target B responder ready
RX DATA seq=0 rssi=-40 snr=9
RX DATA seq=1 rssi=-40 snr=10
RX DATA seq=2 rssi=-40 snr=9
...
```

Optional diagnostic logging is acceptable

Optional extra logging is allowed and may be useful during bring-up. For example, your firmware may also print a board-identifier line such as `UID: ... -> Board X` at startup, and it may print periodic summary lines such as `# stats: sent=... acked=...`. Those lines do *not* change the required radio behavior. If you manually reboot boards or restart monitoring while capturing logs, you may also see repeated startup banners and the sequence counter restarting at zero. That is acceptable for debugging; for the Step 6 analysis, extract one continuous run of at least 100 `CSV,...` rows from a single steady-state session.

[Figure placeholder]

`LH1D_two_node_bench.jpg`

Both Nucleo+RFM95W nodes on the bench ≈1 m apart with spring antennas upright. Target A (initiator) on the left, Target B (responder) on the right.

Figure 1: Both Nucleo+RFM95W nodes on the bench ≈1 m apart with spring antennas upright. Target A (initiator) on the left, Target B (responder) on the right.

[Figure placeholder]

`LH1D_dual_serial.png`

Side-by-side Serial Monitors: Target A's per-second `CSV` ping-pong log on the left and Target B's matching `RX DATA` log on the right. Sequence numbers must match across the two windows.

Figure 2: Side-by-side Serial Monitors: Target A's per-second `CSV` ping-pong log on the left and Target B's matching `RX DATA` log on the right. Sequence numbers must match across the two windows.

4.6 Step 6 — Capture 100 ping-pong pairs and analyze

How to interpret RSSI and SNR

For the SX1276 LoRa receiver, the two link-quality numbers used in this lab are RSSI and SNR.
In real RF terms, RSSI is just received power. The link-budget form is

$$P_{\text{rx,dBm}} = P_{\text{tx,dBm}} + G_{\text{tx,dBi}} + G_{\text{rx,dBi}} - L_{\text{path,dB}} - L_{\text{other,dB}}$$

where transmit power, antenna gains, path loss, and other losses determine how much signal power arrives at the receiver. More positive (or less negative) values indicate a stronger received signal. For example, $-35\,\text{dBm}$ is stronger than $-50\,\text{dBm}$.
SNR is signal power relative to the noise floor. In real-world form,

$$SNR_{\text{dB}} = P_{\text{signal,dBm}} - P_{\text{noise,dBm}}$$

with the thermal-noise estimate

$$P_{\text{noise,dBm}} = -174\,\text{dBm/Hz} + 10\log_{10}(B) + NF$$

where B is receiver bandwidth in Hz and NF is receiver noise figure in dB. Positive SNR means the signal is above the noise floor; negative SNR means the radio is receiving a signal buried below the noise floor but still potentially decodable because LoRa processing gain is high.
In firmware, the SX1276 estimates these values from its packet-status registers. For the high-frequency port used by the RFM95W around 906.5 MHz, the code in this lab converts the register values as

$$RSSI_{\text{dBm}} = \texttt{RegPktRssiValue} - 157$$

and

$$SNR_{\text{dB}} = \frac{\texttt{RegPktSnrValue}}{4}$$

These are not separate physics formulas; they are the SX1276's mapping from its internal measurements to estimated real-world units.
These two numbers are related but not interchangeable:

- RSSI tells you *how much signal power arrived* at the receiver.
- SNR tells you *how clean that signal is relative to noise.*
- A link can have low RSSI but still work well if SNR remains comfortably positive.
- A link can have fairly strong RSSI and still perform poorly if the noise floor rises and SNR falls.

In this handout, RSSI is used for the link-budget comparison, while SNR is a quick indicator of demodulation margin and channel cleanliness.

1. In the Target A Serial Monitor, capture at least 100 consecutive `CSV,...` lines:
   (a) Right-click in the Serial Monitor output area → **Select All** → **Copy** (or press `Ctrl+A` then `Ctrl+C`).
   (b) Paste into a text editor and save as `LH1D_pingpong_log.csv`. Strip the non-CSV lines (for example `UID: ...`, `RegVersion`, `lora_init done`, `Target A initiator ready`, periodic `# stats` lines, and any repeated startup block caused by a manual reset or monitor restart) so only one continuous set of `CSV,seq,t_tx_ms,t_ack_ms,rtt_ms,rssi_dbm` rows remains. Add a header row.

2. From `LH1D_pingpong_log.csv`, compute:
   - **ACK reception rate**: rows with `t_ack_ms` $\neq 0$ divided by total rows. Target: $\geq$95%.
   - **Round-trip latency statistics**: mean, standard deviation, minimum, maximum, and 99th-percentile of `rtt_ms` across rows where `t_ack_ms` $\neq 0$. With the simple polling implementation used in LH1-D, a steady-state mean around ∼140–160 ms is normal at SF7 with a 32-byte payload.
   - **Mean RSSI** and **standard deviation of RSSI** across all rows with valid ACK.
   - **Sequence gaps**: any jump in `seq` indicates a dropped DATA on Target B; any `t_ack_ms` $= 0$ with later sequence numbers continuing indicates a dropped ACK on Target A.

3. Compare mean RSSI against the Module 1 link budget estimate for 1 m indoor separation. For Friis free-space path loss at 906.5 MHz over 1 m:

$$L_{\text{fs}} = 20\log_{10}\left(\frac{4\pi d f}{c}\right) \approx 31.6\,\text{dB}$$

   With +14 dBm transmit power and ~ -2 dBi antenna efficiency on each end, expected received power ≈ -22 dBm. Indoor multipath will shift this; a measured value between -30 and -50 dBm at 1 m is typical. A discrepancy >10 dB warrants an antenna inspection.

5 Verification Checklist

- ☐ Both `Nucleo_RFM95W_TargetA` and `Nucleo_RFM95W_TargetB` projects build with zero errors.
- ☐ `lora_init done` prints on both targets at startup.
- ☐ Target B prints `Target B responder ready` before Target A is started.
- ☐ Target A's `CSV` log shows matching sequence numbers and non-zero `t_ack_ms` for ≥95% of rows.
- ☐ Target B's `RX DATA seq` log matches Target A's outgoing sequence numbers one-for-one.
- ☐ Round-trip latency mean is between 120 and 180 ms for the polling-based LH1-D implementation at SF7.
- ☐ Zero CRC errors on Target B across 100 consecutive DATA receptions.
- ☐ Mean RSSI is within ±10 dB of the computed link budget.

Troubleshooting

- **No ACKs received on Target A**: confirm Target B's `Modulation Code` byte for ACK is `(MOD_KIND_LORA ≪ 2) | PKT_TYPE_ACK` = `0x05`, not bare `0x01`. The packed type field is the most common bug introduced in Step 2.
- **Target A logs all-zero ACKs**: Target A's RX-window timeout (200 ms) is too short for the round-trip at the current SF/payload. In the simple polling implementation used here, measured round-trip latency is often about 150 ms at SF7 with 32-byte packets, so a 200 ms window is only modestly above the observed nominal value. Widen the window to 300 ms if Target B is verifiably transmitting.
- **Target B prints `CRC error` on every packet**: verify `CrcOn` is set (`RegModemConfig2 = 0x74`) on the Target A side and that the preamble length (`0x08`) matches.
- **Sequence gaps on Target B**: usually transient half-duplex collisions if Target B failed to switch back into RX before the next Target A DATA. Confirm that `rfm_set_mode_rx()` is called at the end of the ACK path before the loop iterates.
- **Very low RSSI (< -90 dBm at 1 m)**: usually an antenna problem — re-check the spring antenna solder joint on both modules (LH1-A).
- **Build fails after the LH1-C demo code is removed**: stale CMake cache. Delete the `build/` folder in the project root and press `F7` again.

Lab Handout 1-D Deliverable

1. Source code for both `Nucleo_RFM95W_TargetA` and `Nucleo_RFM95W_TargetB` projects (or just the modified `main.c` files), including `lora_init()`, `build_packet()`, `crc16_ccitt()`, and the role-specific main loops.
2. Captured `LH1D_pingpong_log.csv` containing at least 100 ping-pong pairs.
3. Summary statistics: ACK reception rate, mean / std / p99 of round-trip latency, mean RSSI, RSSI standard deviation, sequence-gap count.
4. One-paragraph comparison between measured mean RSSI and the Module 1 link-budget prediction. Explain any discrepancy >10 dB.

Glossary

CRC	Cyclic Redundancy Check	3, 17
FSM	Finite State Machine	1, 2
RF	Radio Frequency	1
RSSI	Received Signal Strength Indicator	1, 15, 16, 17, 18
SNR	Signal-to-Noise Ratio	1, 15